

Programming and Algorithms

Aula 13: Operações Básicas em Sequências

Prof. Ettore Baldini-Neto
ettore.neto@faculdadeimpacta.com.br

Aula Passada

- Introdução às sequências
  - tipos de sequência
  - percorrendo sequências
- O laço for
  - Exemplos e exercícios

Programa da Aula

- Métodos e objetos em Python
- Operações comuns de sequências
- Laços implícitos
- Operações de sequências mutáveis.
- Substituição de itens na sequência
- Inclusão e Remoção de itens na sequência

Programa da Aula

- Inversão de sequência
- Operações comuns em strings
- Desempacotamento de sequências
- Tipos de passagens de argumentos
- Passagem de argumentos em Python

Introdução

- A ideia é manipular dados em sequências de maneira eficaz.
- Para isso é fundamental entender quais as operações disponíveis para **todas as sequências** e quais são as operações **específicas** para um **dado tipo de sequência.**
- A ideia é ter pelo menos uma noção do que são **objetos** e como a linguagem trata esses objetos criados na memória.
- Muitas das operações realizadas com sequências são feitas através de **métodos** e, como dito, é importante entender minimamente esses conceitos de objetos, métodos e como se relacionam no Python.
- Vamos estudar ainda o comportamento dos objetos quando passados como argumentos para funções integradas, importadas ou criadas.

Métodos e Objetos em Python

- Em Python tudo é objeto! Desde números, passando por listas, até funções.
- Vamos entender:
  - **O que são métodos?**
  - **Como os objetos se comportam quando passados como argumentos para funções?**
- Um objeto agrupa características e comportamentos. Os comportamentos são o que chamamos de métodos e são análogos às funções.
  - **Cada objeto possui uma identidade em Python, como um RG ou CPF**
  - **Podemos usar métodos de um objeto (dependendo do tipo) para acessar ou alterar suas características.**

Métodos e Objetos em Python

- **Função id** (integrada): retorna um número interno que representa a identidade do objeto (representa o endereço do objeto na memória e é único enquanto ele existir)
- **Os métodos** podem ser acessados a partir do **objeto** usando a notação de ponto, ou seja, objeto.metodo( )

```
>>> texto = 'Olá {}!'
>>> id(texto)
3124235308912
>>> texto
'Olá {}!'
>>> texto.format('mundo')
'Olá mundo!'
```

→ objeto.metodo('mundo ')

Métodos e Objetos em Python

```
>>> texto = 'Olá {}!'
>>> id(texto)
3124235308912
>>> texto
'Olá {}!'
>>> texto.format('mundo')
'Olá mundo!'
```

O **método** format foi utilizado para acessar o conteúdo do **objeto** (string) através da referência a ele que foi salva na variável texto. Ele retorna uma nova string formatada (exibida no Shell). Vejam que há a criação de uma nova string pois strings são imutáveis (não podem ser alteradas).

```
>>> id(texto)
3124235308912
>>> texto
'Olá {}!'
```

- **Note:** A variável texto **continua** referenciando a string inicial.
- **Note**: O objeto **não foi modificado** pelo format.

Métodos e Objetos em Python

Vimos que listas são objetos mutáveis. Ou seja, suas características podem ser alteradas no mesmo local da memória, sem a necessidade de gerar outro objeto, diferentemente das strings, como acabamos de ver.

Logo, podemos modificar seus itens, ou mesmo incluir um novo item no final de uma lista utilizando o **método append**. Em ambas as situações a lista continua sendo o mesmo objeto na memória, só que com valores diferentes.

Resumo: Podemos alterar suas características mas não sua identidade.

Métodos e Objetos em Python

Exemplos

```
>>> lista = [1, 2, 3]
>>> lista
[1, 2, 3]
>>> id(lista)
3124235308003
>>> lista[0] = 25
>>> lista
[25, 2, 3]
>>> lista.append(4)
>>> lista
[25, 2, 3, 4]
>>> id(lista)
3124235308003
```

Operações comuns em sequências

São aquelas que podem ser aplicadas a **todos** os tipos de sequências, independente de serem mutáveis ou não. Logo, essas operações se aplicam às listas, tuplas, intervalos e strings. São elas:

- Pertencimento
- Concatenação e repetição
- Indexação e fatiamento
- Tamanho, item mínimo e item máximo
- Busca por valor
- Contagem de ocorrências

Operações comuns em sequências

Pertencimento

- Verifica se um item pertence a uma sequência.
- É feita com o operador **in**
- A operação inversa pode ser feita com a negação, ou seja, **not in**

```
>>> 3 in [1, 2, 5]
False
>>> 5 in range(10)
True
>>> 'z' not in 'banana'
True
```

Operações comuns em sequências

Pertencimento

Para strings há um funcionamento especial deste operador.

Pode ser usado para testes de subsequências.

```
>>>
>>> 'ana'in 'Adriana'
True
>>> 'eta' not in 'Tieta'
False
>>>
```

Operações comuns em sequências

Concatenação

A concatenação é feita com o operador **+.** Note que esta operação **não é aplicável aos intervalos (range)**

```
>>> (1, 2) + (3, 4)
(1, 2, 3, 4)
>>> 'abc' + 'xyz'
'abcxyz'
>>> [30, 40] + [10, 20, 50]
[30, 40, 10, 20, 50]
```

Operações comuns em sequências

Repetição

A operação de repetição é feita com operador ***. Ela também não pode ser aplicada aos intervalos.**

```
>>> 3 * 'Abc'
'AbcAbcAbc'
>>> 5 * [0]
[0, 0, 0, 0, 0]
>>> 4 * (123,)
(123, 123, 123, 123)
```

Operações comuns em sequências

Note que: Concatenação e Repetição **não alteram os objetos, geram novos!**

E por isso, **não são eficientes** para adicionar itens a uma sequência. Uma maneira melhor de fazer isso é utilizar o método **append** em uma lista e, em seguida, convertê-la para outro tipo de sequência, caso seja necessário. Estas duas operações passam para a nova sequência as referências aos itens das sequências iniciais, e não fazem uma cópia do objeto.

```
>>>
>>> s = [[2]] * 3
>>> s
[[2], [2], [2]]
>>> s[0][0] = 5
>>> s
[[5], [5], [5]]
>>>
```

Criamos uma lista com 3 sub-listas contendo o número 2.

Ao alterarmos o primeiro item da primeira sublista, as demais também são alteradas.

Operações comuns em sequências

Indexação

- É a forma como acessamos um item de uma sequência
- É feita sempre com o uso de colchetes [ ]
- Os índices devem ser obrigatoriamente números inteiros e já vimos que há dois tipos de índices: os naturais e os negativos.

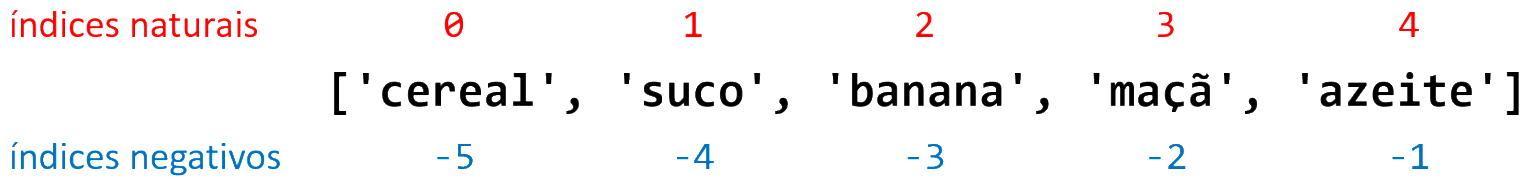


Figura 10.1: Visualização dos índices de uma lista de *strings* em Python.
Fonte: Elaborado pelo autor.

Operações comuns em sequências

Indexação

```
>>> texto = 'uma frase qualquer'
>>> texto[4]
'f'
>>> intervalo = range(10, 20)
>>> intervalo[3]
13
>>> tupla = (3.1, 5.8, 7.0)
>>> tupla[0]
3.1
```

```
>>>
>>> 'abc'[0]
'a'
>>> range(2,5)[1]
3
>>> [40,50,60][2]
60
>>>
```

Operações comuns em sequência

Fatiamento

Ocorre quando definimos um intervalo de índices entre colchetes [ ] para obter uma fatia da sequência, ou seja, uma subsequência.

Para realizar o fatiamento devemos passar os índices separados por dois pontos entre os colchetes:

Sintaxe: sequencia[inicio:fim:passo]

Um fatiamento **não altera a sequência fatiada**, apenas gera uma nova sequência **do mesmo tipo da original** contendo apenas os itens (cópias) conforme a definição dada. Novamente o item **fim** não é incluído no fatiamento. Vamos ver alguns exemplos com listas mas que **valem para outros tipos de sequências**.

Operações comuns em sequências

```
>>> lista = [30, 12, 13, 41, 15, 6, 78, 44, 19]
>>> lista[2:4]     # se omitido, passo é 1
[13, 41]
>>> lista[1::3]    # se omitido, fim é len(lista)
[12, 15, 44]
>>> lista[:3]      # se omitido, início é 0
[30, 12, 13]
>>> lista[5:1]     # se fim <= início, a fatia é vazia
[]
>>> lista[5:1:-1]  # se passo é negativo, inverte a ordem da fatia
[6, 15, 41, 13]
>>> lista[:]       # fatia correspondente à lista inteira
[30, 12, 13, 41, 15, 6, 78, 44, 19]
```

Operações comuns em sequências

Tamanho, soma, mínimo e máximo

Tabela 10.1: Funções integradas `len`, `sum`, `min` e `max`.

Função	Descrição
`len(s)`	Retorna o comprimento (tamanho) da sequência `s`.
`sum(s)`	Retorna a soma dos itens da sequência `s`.
`min(s)`	Retorna o item de valor mínimo da sequência `s`.
`max(s)`	Retorna o item de valor máximo da sequência `s`.

Operações comuns em sequências

```
>>> lista = [30, 12, 13, 41, 15, 6, 78, 44, 19]
>>> len(lista)
9
>>> len(range(3, 10, 2))
4
>>> sum(lista)
258
>>> max(lista)
78
>>> min('Python')   # Por que a menor letra é  'P' e não 'h'?[1]
'P'
```

(3, 5, 7, 9)

[1] Os caracteres são comparados por seus códigos na tabela Unicode e, neste caso, letras latinas maiúsculas antecedem as minúsculas. Para recordar basta usar as funções `ord(caractere)` e `chr(código)`.

Operações comuns em sequências

Busca por valor

Para buscar o índice de um valor em uma sequência, pode-se utilizar o método **index. Este método recebe como argumento o valor buscado e retorna a posição da primeira ocorrência encontrada.**

```
>>> texto = 'Eu não aguento mais estudar!'
>>> texto.index('ã')
4
>>> texto.index('u')
1
>>> texto.index('I')
Traceback (most recent call last):
  File "<pyshell#3>", line 1, in <module>
    texto.index('I')
ValueError: substring not found
>>>
```

Operações comuns em sequências

Contagem de ocorrências

Para contar quantas vezes um valor ocorre em uma sequência, usamos o método **count.** Ele recebe o valor buscado e percorre a sequência inteira, retornando no final quantas vezes aquele valor foi encontrado.

```
>>> lista = [1, 2, 3, 41, 5, 7, 3, 41, 41]
>>> lista.count(41)
3
>>> lista.count(200)
0
```

Laços implícitos

Várias funções e métodos aplicados em sequências precisam percorrer a sequência para fazerem seu trabalho. Isso mostra que há laços implícitos ocorrendo, mas são invisíveis pois não temos acesso ao código dessas funções e métodos.

Deste modo, há um custo computacional (tempo e processamento de memória) que fica mais evidente quando as listas a serem percorridos são muito grandes.

Operações com sequências mutáveis

Nas sequências mutáveis existem mais operações possíveis. As que veremos, a partir de agora, aplicam-se apenas às listas.

- Substituição de itens da sequência
- Inclusão de itens na sequência
- Remoção de itens da sequência
- Inversão de sequência

Operações com sequências mutáveis

Substituição de itens da sequência

Feita por atribuição direta.

```
>>> lista = [41, 5, 7]
>>> lista[1] = 200
>>> lista
[41, 200, 7]
```

Inclusão de itens na sequência

Há três formas de se fazer isso, dependendo da localização e quantidade de itens que devem ser incluídos.

Operações com sequências mutáveis

- `s.insert(i, x)` - insere `x` como item na posição `i` da sequência `s`;
- `s.append(x)` - adiciona `x` como item no final da sequência `s`;
- `s.extend(t)` - adiciona, um a um, os itens da sequência `t` ao final de `s`.

```
>>> lista = [5, 3, 78]
>>> lista.insert(1, 25)
>>> lista
[5, 25, 3, 78]
>>>
>>> lista.append(45)
>>> lista
[5, 25, 3, 78, 45]
>>>
>>>
>>> lista.extend((1,2,3))
>>> lista
[5, 25, 3, 78, 45, 1, 2, 3]
>>>
```

Operações com sequências mutáveis

Remoção de itens na sequência

Há três formas de se remover um item de uma sequência:

- `del s[i]` - exclui o item de índice `i` na sequência `s`;
- `s.pop(i)` - exclui o item de índice `i` na sequência `s`, e retorna seu valor;
- `s.remove(x)` - remove o primeiro item de valor `x` da sequência `s`.

```
>>>
>>> lista = [15, 10, 99]
>>> del lista[2]
>>> lista
[15, 10]
>>>
```

```
>>>
>>> lista = [15, 10, 99]
>>> lista.pop(0)
15
>>>
```

```
>>> lista = [15, 10, 99]
>>> lista.remove(10)
>>> lista
[15, 99]
>>>
```

Operações com sequências mutáveis

Note que: com **del** e **pop**, o item é diretamente acessado por meio de seu índice, logo o custo computacional é constante**.** Já com o método **remove**, a lista é percorrida até que o valor desejado seja encontrado ou ela termine**.** Logo, neste caso, há um laço implícito e o custo cresce proporcionalmente ao tamanho da lista**.**

Inversão de sequência: método **reverse**

```
>>> lista = [10, 20, 30, 40]
>>> lista.reverse()
>>> lista
[40, 30, 20, 10]
```

Operações comuns em strings

Para separar uma string em uma lista de strings usamos o método **split.**

- **Ele recebe um separador (string) e retorna uma lista.**
- **Exemplo:**

```
>>> '1;2;3;4;5'.split(';')
['1', '2', '3', '4', '5']
>>> 'a.b.c.d.e.f.g'.split('.')
['a', 'b', 'c', 'd', 'e', 'f', 'g']
>>>
```

Operações comuns em strings

Caso o método seja chamado **sem argumentos**, ele vai separar a string em todos os caracteres que representam **espaços em branco** (espaços, tabulação e quebra de linha) e eliminando espaços em branco consecutivos.

```
>>> 'frase  com     espaços  '.split(' ')
['frase', '', 'com', '', '', '', '', 'espaços', '', '']
>>> 'frase  com     espaços  '.split()
['frase', 'com', 'espaços']
```

- Ao passarmos um espaço em branco como argumento, o Python coloca **uma string vazia entre cada espaço adjacente na string** (linha 1).
- Quando não passamos argumento ele automaticamente **desconsidera** espaços extras (linha 3)

Operações comuns em strings

O **método join** faz o inverso, ou seja, ele recebe um argumento como uma sequência de strings e gera uma nova string com os itens do argumento **concatenados e separados por caracteres de string com os quais foi chamado.**

```
>>>
>>> '_'.join(['1', '2', '3', '4', '5'])
'1_2_3_4_5'
>>>
>>> '_sep_'.join(['a','b', 'c'])
'a_sep_b_sep_c'
>>>
>>> ''.join(['a','b', 'c'])
'abc'
>>> ' '.join(['a','b', 'c'])
'a b c'
>>>
```

Desempacotamento de sequências

O desempacotamento ocorre quando os itens de uma sequência são atribuídos a diversas variáveis em uma única atribuição (atribuição paralela)

```
>>> x, y, z = [15, 10, 99]
>>> print(f'x = {x} | y = {y} | z = {z}')
x = 15 | y = 10 | z = 99
>>> a, b, c = (10, 'Impacta', True)
>>> print(f'a = {a} | b = {b} | c = {c}')
a = 10 | b = Impacta | c = True
```

Codificação 10.27: Desempacotamento de uma sequência em variáveis.

Se o número de variáveis à esquerda for diferente do número de itens à direita um erro será gerado!

Desempacotamento de sequências

Uma maneira simples de realizar a leitura de diversos valores dispostos em uma única linha é com a **combinação** do método split e do desempacotamento.

```
nome, idade, peso = input().split()     # desempacotamento de lista.
idade, peso = int(idade), float(peso)   # desempacotamento de tupla.
```

Codificação 10.28: Leitura de diversos valores dispostos em apenas uma linha.

Note que a variável nome já tem seu valor correto (string) no primeiro desempacotamento e não precisou de uma nova atribuição, ao contrário de idade e peso que tiveram que ser especificadas.

Desempacotamento de sequências

Exemplo

```
def teste(a, b, c):
    print(f'a = {a} | b = {b} | c = {c}')

lista = [1, 'banana', True]
teste(*lista)
```

```
==================== RESTART: /Users/Ettore/Desktop/teste_f.py =====
a = 1 | b = banana | c = True
>>
```

Chamando a função no shell temos.

```
>>> teste(lista[0], lista[1], lista[2])
a = 1 | b = banana | c = True
>>>
```

Desempacotamento de sequências

- A função print tem uma quantidade variável de argumentos.
- Podemos exibir todos os itens de uma lista só com a instrução

print(*lista)

```
>>> lista1 = ['abacaxi', 'banana', 'laranja']
>>> print(*lista1)
abacaxi banana laranja
>>> lista2 = [1,3,5,7,9,11,13]
>>> print(*lista2)
1 3 5 7 9 11 13
>>>
```

Passagem de argumentos/parâmetros

Quando chamamos uma função há duas alternativas para passagem dos argumentos:

- **passando o valor diretamente**: Aqui, o parâmetro da função recebe uma cópia do valor original
- **passando uma referência** para o local da memória onde o valor está. Assim, o parâmetro da função receberá o endereço do valor original

FIM